

Grammarcetamol — Database Schema & Migration Scripts

Document Version: 1.0

Last Updated: 2026-08-01

Relational DB: PostgreSQL 15+ (per-service)

Document DB: MongoDB 6+ (Media, Analytics, Live Class, Service Request)

1. Migration Philosophy

Aspect	Relational (PostgreSQL)	Document (MongoDB)
Migration Style	Versioned SQL files with idempotent DDL (IF NOT EXISTS, IF EXISTS)	Schema-validation in app + index-migration scripts
Tooling	Flyway, Liquibase, or raw SQL boot-migration	Mongoose Migrations, mongo-migrate, or custom Node scripts
Safety	Transactions wrap DDL where supported; destructive changes require manual gates	No DDL locks; additive-only in production
Rollback	Down-migration scripts provided per-version	Snapshot + restore for major changes

Note: MongoDB does not use `CREATE TABLE`. Instead, collections are created implicitly. “Migrations” in MongoDB typically mean: (1) creating indexes, (2) setting validation rules, (3) back-filling documents, or (4) renaming fields via update scripts.

2. Service-to-Database Mapping

Microservice	Database	Engine	Justification
Auth Service	auth_db	PostgreSQL	ACID transactions, strict consistency for credentials
User Service	user_db	PostgreSQL	Relational user/profile/role data
Course Service	course_db	PostgreSQL	Structured content with heavy referential integrity
Enrollment Service	enrollment_db	PostgreSQL	Transactional enrollments & progress tracking
Payment Service	payment_db	PostgreSQL	Financial records require ACID compliance
Review Service	review_db	PostgreSQL	Structured ratings with aggregations
Notification Service	notification_db	PostgreSQL	Ordered, queryable message logs

Table 2 – continued

Microservice	Database	Engine	Justification
Admin Service	admin_db	PostgreSQL	Audit trails, settings, RBAC
Live Class Service	liveclass_db	MongoDB	Flexible session metadata, rapid schema evolution
Media Service	media_db	MongoDB	Unstructured asset metadata, transcoding state machines
Analytics Service	analytics_db	MongoDB	High-write event streams, time-series friendly
Service Request Service	request_db	MongoDB	Evolving request forms, nested conversation threads
Upload Service	upload_db	PostgreSQL	Chunked upload session state (high consistency)

3. PostgreSQL Migrations (Idempotent)

3.1 Auth Service — auth_db

```
-- =====
-- Migration: V1_auth_initial_schema.sql
-- Service: Auth Service
-- =====

-- Users table (canonical identity)
CREATE TABLE IF NOT EXISTS users (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email       VARCHAR(255) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  status      VARCHAR(20) NOT NULL DEFAULT 'pending_verification'
              CHECK (status IN ('pending_verification', 'active', 'suspended', 'deleted')),
  email_verified BOOLEAN NOT NULL DEFAULT FALSE,
  failed_attempts INT NOT NULL DEFAULT 0,
  locked_until TIMESTAMPTZ,
  last_login_at TIMESTAMPTZ,
  created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at   TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS idx_users_email ON users(email);
CREATE INDEX IF NOT EXISTS idx_users_status ON users(status);
CREATE INDEX IF NOT EXISTS idx_users_locked_until ON users(locked_until) WHERE locked_until IS NOT NULL;

-- Refresh tokens (rotating)
CREATE TABLE IF NOT EXISTS refresh_tokens (
```

```

    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id            UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    token_hash         VARCHAR(255) NOT NULL UNIQUE,
    device_info        VARCHAR(255),
    ip_address         INET,
    expires_at         TIMESTAMPTZ NOT NULL,
    revoked_at         TIMESTAMPTZ,
    created_at         TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS idx_refresh_tokens_user_id ON refresh_tokens(user_id);
CREATE INDEX IF NOT EXISTS idx_refresh_tokens_token_hash ON refresh_tokens(token_hash);
CREATE INDEX IF NOT EXISTS idx_refresh_tokens_expires_at ON refresh_tokens(expires_at);

-- JWT blacklist (for logout / security events)
CREATE TABLE IF NOT EXISTS jwt_blacklist (
    jti                UUID PRIMARY KEY,
    user_id            UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    expires_at         TIMESTAMPTZ NOT NULL,
    reason             VARCHAR(50) NOT NULL DEFAULT 'logout',
    created_at         TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS idx_jwt_blacklist_expires_at ON jwt_blacklist(expires_at);

-- Trigger function for updated_at
CREATE OR REPLACE FUNCTION update_updated_at_column()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = NOW();
    RETURN NEW;
END;
$$ language 'plpgsql';

DO $$
BEGIN
    IF NOT EXISTS (SELECT 1 FROM pg_trigger WHERE tgname = 'trg_users_updated_at') THEN
        CREATE TRIGGER trg_users_updated_at
        BEFORE UPDATE ON users
        FOR EACH ROW EXECUTE FUNCTION update_updated_at_column();
    END IF;
END $$;

```

3.2 User Service — user_db

```

-- =====
-- Migration: V1__user_initial_schema.sql
-- Service: User Service
-- =====

```

```

-- Roles (RBAC)
CREATE TABLE IF NOT EXISTS roles (
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name          VARCHAR(50) NOT NULL UNIQUE,
    description    TEXT,
    is_system     BOOLEAN NOT NULL DEFAULT FALSE,
    created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Insert default roles idempotently
INSERT INTO roles (id, name, description, is_system)
VALUES
    ('11111111-1111-1111-1111-111111111111', 'super_admin', 'Full platform access', TRUE),
    ('22222222-2222-2222-2222-222222222222', 'moderator', 'Configurable operational access', TRUE),
    ('33333333-3333-3333-3333-333333333333', 'student', 'Learner account', TRUE),
    ('44444444-4444-4444-4444-444444444444', 'instructor', 'Content creator', TRUE)
ON CONFLICT (name) DO NOTHING;

-- User profiles (extends auth identity)
CREATE TABLE IF NOT EXISTS user_profiles (
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    auth_user_id  UUID NOT NULL UNIQUE, -- mirrors auth service user id
    full_name     VARCHAR(255) NOT NULL,
    phone        VARCHAR(30),
    avatar_url    VARCHAR(500),
    country       VARCHAR(2), -- ISO-3166
    timezone      VARCHAR(50) NOT NULL DEFAULT 'UTC',
    bio          TEXT,
    learning_goals TEXT[],
    date_of_birth DATE,
    preferences   JSONB NOT NULL DEFAULT '{}',
    created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS idx_user_profiles_auth_user_id ON user_profiles(auth_user_id);
CREATE INDEX IF NOT EXISTS idx_user_profiles_country ON user_profiles(country);

-- User-Role mapping (many-to-many)
CREATE TABLE IF NOT EXISTS user_roles (
    user_id       UUID NOT NULL REFERENCES user_profiles(id) ON DELETE CASCADE,
    role_id       UUID NOT NULL REFERENCES roles(id) ON DELETE CASCADE,
    assigned_by   UUID,
    assigned_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    PRIMARY KEY (user_id, role_id)
);

CREATE INDEX IF NOT EXISTS idx_user_roles_role_id ON user_roles(role_id);

-- Permissions matrix (granular for moderators)

```

```

CREATE TABLE IF NOT EXISTS permissions (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  resource    VARCHAR(50) NOT NULL, -- e.g., 'courses', 'students'
  action      VARCHAR(50) NOT NULL, -- e.g., 'create', 'edit', 'delete'
  UNIQUE (resource, action)
);

CREATE TABLE IF NOT EXISTS role_permissions (
  role_id      UUID NOT NULL REFERENCES roles(id) ON DELETE CASCADE,
  permission_id UUID NOT NULL REFERENCES permissions(id) ON DELETE CASCADE,
  PRIMARY KEY (role_id, permission_id)
);

CREATE INDEX IF NOT EXISTS idx_role_permissions_role_id ON role_permissions(role_id);

-- Addresses (for billing / invoicing)
CREATE TABLE IF NOT EXISTS user_addresses (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID NOT NULL REFERENCES user_profiles(id) ON DELETE CASCADE,
  type        VARCHAR(20) NOT NULL DEFAULT 'billing' CHECK (type IN ('billing', 'shipping')),
  street      VARCHAR(255) NOT NULL,
  city        VARCHAR(100) NOT NULL,
  state       VARCHAR(100),
  postal_code VARCHAR(20),
  country     VARCHAR(2) NOT NULL,
  is_default  BOOLEAN NOT NULL DEFAULT FALSE,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS idx_user_addresses_user_id ON user_addresses(user_id);

-- Triggers
DO $$
BEGIN
  IF NOT EXISTS (SELECT 1 FROM pg_trigger WHERE tgname = 'trg_user_profiles_updated_at') THEN
    CREATE TRIGGER trg_user_profiles_updated_at
    BEFORE UPDATE ON user_profiles
    FOR EACH ROW EXECUTE FUNCTION update_updated_at_column();
  END IF;
END $$;

```

3.3 Course Service — `course_db`

```

-- =====
-- Migration: V1__course_initial_schema.sql
-- Service: Course Service
-- =====

-- Categories

```

```

CREATE TABLE IF NOT EXISTS categories (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name        VARCHAR(100) NOT NULL UNIQUE,
  slug        VARCHAR(100) NOT NULL UNIQUE,
  description  TEXT,
  icon_url    VARCHAR(500),
  parent_id   UUID REFERENCES categories(id) ON DELETE SET NULL,
  sort_order  INT NOT NULL DEFAULT 0,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS idx_categories_slug ON categories(slug);
CREATE INDEX IF NOT EXISTS idx_categories_parent_id ON categories(parent_id);

-- Courses
CREATE TABLE IF NOT EXISTS courses (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  instructor_id  UUID NOT NULL, -- references user_profiles.id
  title        VARCHAR(255) NOT NULL,
  subtitle     VARCHAR(500),
  description   TEXT NOT NULL,
  learning_objectives TEXT[] NOT NULL DEFAULT '{}',
  target_audience TEXT,
  prerequisites TEXT,
  category_id  UUID REFERENCES categories(id) ON DELETE SET NULL,
  difficulty   VARCHAR(20) NOT NULL CHECK (difficulty IN ('beginner', 'intermediate',
    ↪ 'advanced')),
  language     VARCHAR(10) NOT NULL DEFAULT 'en',
  estimated_duration INT, -- minutes
  status       VARCHAR(20) NOT NULL DEFAULT 'draft'
    CHECK (status IN ('draft', 'review', 'published', 'archived')),
  price        DECIMAL(10,2) NOT NULL DEFAULT 0.00,
  discount_price DECIMAL(10,2),
  discount_expires_at TIMESTAMPTZ,
  currency     VARCHAR(3) NOT NULL DEFAULT 'USD',
  cover_image_url VARCHAR(500),
  promo_video_url VARCHAR(500),
  enrollment_count INT NOT NULL DEFAULT 0,
  avg_rating    DECIMAL(2,1) CHECK (avg_rating >= 0 AND avg_rating <= 5),
  review_count  INT NOT NULL DEFAULT 0,
  version       INT NOT NULL DEFAULT 1,
  published_at  TIMESTAMPTZ,
  created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS idx_courses_instructor_id ON courses(instructor_id);
CREATE INDEX IF NOT EXISTS idx_courses_category_id ON courses(category_id);
CREATE INDEX IF NOT EXISTS idx_courses_status ON courses(status);
CREATE INDEX IF NOT EXISTS idx_courses_difficulty ON courses(difficulty);
CREATE INDEX IF NOT EXISTS idx_courses_price ON courses(price);

```

```

CREATE INDEX IF NOT EXISTS idx_courses_published_at ON courses(published_at) WHERE published_at IS NOT
↪ NULL;
CREATE INDEX IF NOT EXISTS idx_courses_search ON courses USING gin(to_tsvector('english', title || ' '
↪ || COALESCE(subtitle, '') || ' ' || COALESCE(description, '')));

-- Course versions (audit / rollback)
CREATE TABLE IF NOT EXISTS course_versions (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  course_id   UUID NOT NULL REFERENCES courses(id) ON DELETE CASCADE,
  version     INT NOT NULL,
  snapshot    JSONB NOT NULL, -- full course snapshot
  changed_by  UUID NOT NULL,
  change_summary TEXT,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  UNIQUE (course_id, version)
);

CREATE INDEX IF NOT EXISTS idx_course_versions_course_id ON course_versions(course_id);

-- Modules
CREATE TABLE IF NOT EXISTS modules (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  course_id   UUID NOT NULL REFERENCES courses(id) ON DELETE CASCADE,
  title       VARCHAR(255) NOT NULL,
  description  TEXT,
  position     INT NOT NULL DEFAULT 0,
  is_published BOOLEAN NOT NULL DEFAULT TRUE,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS idx_modules_course_id ON modules(course_id);
CREATE INDEX IF NOT EXISTS idx_modules_position ON modules(course_id, position);

-- Lessons
CREATE TYPE lesson_type AS ('video', 'text', 'quiz', 'resource');

CREATE TABLE IF NOT EXISTS lessons (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  module_id   UUID NOT NULL REFERENCES modules(id) ON DELETE CASCADE,
  title       VARCHAR(255) NOT NULL,
  description  TEXT,
  type        lesson_type NOT NULL DEFAULT 'video',
  duration     INT, -- seconds
  position     INT NOT NULL DEFAULT 0,
  video_url    VARCHAR(500),
  video_metadata JSONB, -- resolution, codec, etc.
  is_preview   BOOLEAN NOT NULL DEFAULT FALSE,
  is_published BOOLEAN NOT NULL DEFAULT TRUE,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()

```

```

);

CREATE INDEX IF NOT EXISTS idx_lessons_module_id ON lessons(module_id);
CREATE INDEX IF NOT EXISTS idx_lessons_position ON lessons(module_id, position);
CREATE INDEX IF NOT EXISTS idx_lessons_type ON lessons(type);

-- Resources (lesson attachments)
CREATE TABLE IF NOT EXISTS resources (
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    lesson_id     UUID NOT NULL REFERENCES lessons(id) ON DELETE CASCADE,
    file_name     VARCHAR(255) NOT NULL,
    file_type     VARCHAR(50) NOT NULL, -- pdf, docx, etc.
    file_size     BIGINT NOT NULL,      -- bytes
    file_url      VARCHAR(500) NOT NULL,
    mime_type     VARCHAR(100),
    checksum      VARCHAR(64),
    sort_order    INT NOT NULL DEFAULT 0,
    created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS idx_resources_lesson_id ON resources(lesson_id);

-- Tags
CREATE TABLE IF NOT EXISTS tags (
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name          VARCHAR(50) NOT NULL UNIQUE,
    slug          VARCHAR(50) NOT NULL UNIQUE,
    created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS course_tags (
    course_id     UUID NOT NULL REFERENCES courses(id) ON DELETE CASCADE,
    tag_id        UUID NOT NULL REFERENCES tags(id) ON DELETE CASCADE,
    PRIMARY KEY (course_id, tag_id)
);

CREATE INDEX IF NOT EXISTS idx_course_tags_tag_id ON course_tags(tag_id);

-- Triggers
DO $$
BEGIN
    IF NOT EXISTS (SELECT 1 FROM pg_trigger WHERE tgname = 'trg_courses_updated_at') THEN
        CREATE TRIGGER trg_courses_updated_at BEFORE UPDATE ON courses FOR EACH ROW EXECUTE FUNCTION
            ↪ update_updated_at_column();
    END IF;
    IF NOT EXISTS (SELECT 1 FROM pg_trigger WHERE tgname = 'trg_modules_updated_at') THEN
        CREATE TRIGGER trg_modules_updated_at BEFORE UPDATE ON modules FOR EACH ROW EXECUTE FUNCTION
            ↪ update_updated_at_column();
    END IF;
    IF NOT EXISTS (SELECT 1 FROM pg_trigger WHERE tgname = 'trg_lessons_updated_at') THEN

```

```

CREATE TRIGGER trg_lessons_updated_at BEFORE UPDATE ON lessons FOR EACH ROW EXECUTE FUNCTION
    → update_updated_at_column();
END IF;
END $$;

```

3.4 Enrollment Service — enrollment_db

```

-- =====
-- Migration: V1__enrollment_initial_schema.sql
-- Service: Enrollment Service
-- =====

-- Enrollments
CREATE TYPE enrollment_status AS ('active', 'completed', 'dropped', 'expired');

CREATE TABLE IF NOT EXISTS enrollments (
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id       UUID NOT NULL,
    course_id     UUID NOT NULL,
    status        enrollment_status NOT NULL DEFAULT 'active',
    price_paid    DECIMAL(10,2) NOT NULL DEFAULT 0.00,
    currency      VARCHAR(3) NOT NULL DEFAULT 'USD',
    payment_id    UUID, -- references payment_db.payments.id
    enrolled_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    completed_at  TIMESTAMPTZ,
    expires_at    TIMESTAMPTZ,
    created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE (user_id, course_id)
);

CREATE INDEX IF NOT EXISTS idx_enrollments_user_id ON enrollments(user_id);
CREATE INDEX IF NOT EXISTS idx_enrollments_course_id ON enrollments(course_id);
CREATE INDEX IF NOT EXISTS idx_enrollments_status ON enrollments(status);
CREATE INDEX IF NOT EXISTS idx_enrollments_enrolled_at ON enrollments(enrolled_at);

-- Lesson progress
CREATE TABLE IF NOT EXISTS lesson_progress (
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    enrollment_id UUID NOT NULL REFERENCES enrollments(id) ON DELETE CASCADE,
    lesson_id     UUID NOT NULL, -- references course_db.lessons.id
    status        VARCHAR(20) NOT NULL DEFAULT 'not_started'
        CHECK (status IN ('not_started', 'in_progress', 'completed')),
    watch_position INT NOT NULL DEFAULT 0, -- seconds
    completion_pct DECIMAL(5,2) NOT NULL DEFAULT 0.00,
    completed_at  TIMESTAMPTZ,
    last_accessed_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE (enrollment_id, lesson_id)
)

```

```

);

CREATE INDEX IF NOT EXISTS idx_lesson_progress_enrollment_id ON lesson_progress(enrollment_id);
CREATE INDEX IF NOT EXISTS idx_lesson_progress_lesson_id ON lesson_progress(lesson_id);
CREATE INDEX IF NOT EXISTS idx_lesson_progress_status ON lesson_progress(status);

-- Certificates (future)
CREATE TABLE IF NOT EXISTS certificates (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    enrollment_id     UUID NOT NULL UNIQUE REFERENCES enrollments(id) ON DELETE CASCADE,
    certificate_number VARCHAR(100) NOT NULL UNIQUE,
    issued_at         TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    revoked_at        TIMESTAMPTZ,
    revoked_reason     TEXT,
    pdf_url            VARCHAR(500),
    metadata           JSONB NOT NULL DEFAULT '{}')
);

CREATE INDEX IF NOT EXISTS idx_certificates_enrollment_id ON certificates(enrollment_id);
CREATE INDEX IF NOT EXISTS idx_certificates_number ON certificates(certificate_number);

DO $$
BEGIN
    IF NOT EXISTS (SELECT 1 FROM pg_trigger WHERE tgname = 'trg_enrollments_updated_at') THEN
        CREATE TRIGGER trg_enrollments_updated_at BEFORE UPDATE ON enrollments FOR EACH ROW EXECUTE
            ⇨ FUNCTION update_updated_at_column();
    END IF;
    IF NOT EXISTS (SELECT 1 FROM pg_trigger WHERE tgname = 'trg_lesson_progress_updated_at') THEN
        CREATE TRIGGER trg_lesson_progress_updated_at BEFORE UPDATE ON lesson_progress FOR EACH ROW
            ⇨ EXECUTE FUNCTION update_updated_at_column();
    END IF;
END $$;

```

3.5 Payment Service — payment_db

```

-- =====
-- Migration: V1__payment_initial_schema.sql
-- Service: Payment Service
-- =====

CREATE TYPE payment_status AS ('pending', 'processing', 'completed', 'failed', 'refunded',
    ⇨ 'partially_refunded');
CREATE TYPE payment_method AS ('card', 'mobile_money', 'bank_transfer', 'wallet');
CREATE TYPE transaction_type AS ('payment', 'refund', 'payout', 'fee');

-- Payments (intent-level)
CREATE TABLE IF NOT EXISTS payments (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id           UUID NOT NULL,

```

```

course_id      UUID,
service_request_id UUID,
amount         DECIMAL(12,2) NOT NULL,
currency       VARCHAR(3) NOT NULL DEFAULT 'USD',
status         payment_status NOT NULL DEFAULT 'pending',
payment_method payment_method NOT NULL,
gateway        VARCHAR(50) NOT NULL, -- stripe, paystack, flutterwave
gateway_ref    VARCHAR(255),          -- external transaction ID
gateway_response JSONB,
description    TEXT,
metadata       JSONB NOT NULL DEFAULT '{}',
paid_at        TIMESTAMPTZ,
failed_at      TIMESTAMPTZ,
failure_reason TEXT,
created_at     TIMESTAMPTZ NOT NULL DEFAULT NOW(),
updated_at     TIMESTAMPTZ NOT NULL DEFAULT NOW(),
CONSTRAINT chk_payment_target CHECK (
    (course_id IS NOT NULL AND service_request_id IS NULL) OR
    (course_id IS NULL AND service_request_id IS NOT NULL) OR
    (course_id IS NOT NULL AND service_request_id IS NOT NULL)
)
);

CREATE INDEX IF NOT EXISTS idx_payments_user_id ON payments(user_id);
CREATE INDEX IF NOT EXISTS idx_payments_course_id ON payments(course_id);
CREATE INDEX IF NOT EXISTS idx_payments_status ON payments(status);
CREATE INDEX IF NOT EXISTS idx_payments_gateway_ref ON payments(gateway_ref);
CREATE INDEX IF NOT EXISTS idx_payments_created_at ON payments(created_at);

-- Transactions (ledger entries)
CREATE TABLE IF NOT EXISTS transactions (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    payment_id   UUID NOT NULL REFERENCES payments(id) ON DELETE CASCADE,
    type        transaction_type NOT NULL,
    amount      DECIMAL(12,2) NOT NULL,
    currency     VARCHAR(3) NOT NULL,
    status       VARCHAR(20) NOT NULL DEFAULT 'success' CHECK (status IN ('success', 'failed',
        ⇨ 'pending')),
    gateway_ref  VARCHAR(255),
    gateway_response JSONB,
    processed_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS idx_transactions_payment_id ON transactions(payment_id);
CREATE INDEX IF NOT EXISTS idx_transactions_type ON transactions(type);
CREATE INDEX IF NOT EXISTS idx_transactions_processed_at ON transactions(processed_at);

-- Refunds
CREATE TABLE IF NOT EXISTS refunds (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    payment_id   UUID NOT NULL REFERENCES payments(id) ON DELETE CASCADE,

```

```

amount          DECIMAL(12,2) NOT NULL,
currency        VARCHAR(3) NOT NULL,
reason          TEXT NOT NULL,
status          VARCHAR(20) NOT NULL DEFAULT 'pending' CHECK (status IN ('pending', 'approved',
↪ 'rejected', 'completed')),
processed_by    UUID,
processed_at    TIMESTAMPTZ,
gateway_ref     VARCHAR(255),
created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
updated_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
CONSTRAINT chk_refund_amount CHECK (amount > 0)
);

CREATE INDEX IF NOT EXISTS idx_refunds_payment_id ON refunds(payment_id);
CREATE INDEX IF NOT EXISTS idx_refunds_status ON refunds(status);

-- Invoices
CREATE TABLE IF NOT EXISTS invoices (
  id             UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  payment_id     UUID NOT NULL REFERENCES payments(id) ON DELETE CASCADE,
  invoice_number VARCHAR(100) NOT NULL UNIQUE,
  user_id        UUID NOT NULL,
  subtotal       DECIMAL(12,2) NOT NULL,
  tax_amount     DECIMAL(12,2) NOT NULL DEFAULT 0.00,
  discount_amount DECIMAL(12,2) NOT NULL DEFAULT 0.00,
  total          DECIMAL(12,2) NOT NULL,
  currency       VARCHAR(3) NOT NULL,
  line_items     JSONB NOT NULL,
  billing_details JSONB NOT NULL,
  pdf_url        VARCHAR(500),
  sent_at        TIMESTAMPTZ,
  created_at     TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS idx_invoices_payment_id ON invoices(payment_id);
CREATE INDEX IF NOT EXISTS idx_invoices_user_id ON invoices(user_id);
CREATE INDEX IF NOT EXISTS idx_invoices_number ON invoices(invoice_number);

DO $$
BEGIN
  IF NOT EXISTS (SELECT 1 FROM pg_trigger WHERE tgname = 'trg_payments_updated_at') THEN
    CREATE TRIGGER trg_payments_updated_at BEFORE UPDATE ON payments FOR EACH ROW EXECUTE FUNCTION
    ↪ update_updated_at_column();
  END IF;
  IF NOT EXISTS (SELECT 1 FROM pg_trigger WHERE tgname = 'trg_refunds_updated_at') THEN
    CREATE TRIGGER trg_refunds_updated_at BEFORE UPDATE ON refunds FOR EACH ROW EXECUTE FUNCTION
    ↪ update_updated_at_column();
  END IF;
END $$;

```

3.6 Review Service — review_db

```
-- =====
-- Migration: V1__review_initial_schema.sql
-- Service: Review Service
-- =====

CREATE TYPE review_status AS ('pending', 'approved', 'rejected', 'flagged');

CREATE TABLE IF NOT EXISTS reviews (
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id       UUID NOT NULL,
    course_id     UUID NOT NULL,
    enrollment_id UUID NOT NULL,
    rating        INT NOT NULL CHECK (rating >= 1 AND rating <= 5),
    title         VARCHAR(255),
    comment       TEXT,
    status        review_status NOT NULL DEFAULT 'pending',
    moderated_by  UUID,
    moderated_at  TIMESTAMPTZ,
    moderation_note TEXT,
    is_edited     BOOLEAN NOT NULL DEFAULT FALSE,
    edited_at     TIMESTAMPTZ,
    helpful_count INT NOT NULL DEFAULT 0,
    created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE (user_id, course_id)
);

CREATE INDEX IF NOT EXISTS idx_reviews_course_id ON reviews(course_id);
CREATE INDEX IF NOT EXISTS idx_reviews_user_id ON reviews(user_id);
CREATE INDEX IF NOT EXISTS idx_reviews_status ON reviews(status);
CREATE INDEX IF NOT EXISTS idx_reviews_rating ON reviews(rating);
CREATE INDEX IF NOT EXISTS idx_reviews_created_at ON reviews(created_at);

-- Review helpfulness votes
CREATE TABLE IF NOT EXISTS review_votes (
    review_id     UUID NOT NULL REFERENCES reviews(id) ON DELETE CASCADE,
    user_id       UUID NOT NULL,
    is_helpful    BOOLEAN NOT NULL,
    created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    PRIMARY KEY (review_id, user_id)
);

CREATE INDEX IF NOT EXISTS idx_review_votes_review_id ON review_votes(review_id);

DO $$
BEGIN
    IF NOT EXISTS (SELECT 1 FROM pg_trigger WHERE tgname = 'trg_reviews_updated_at') THEN
        CREATE TRIGGER trg_reviews_updated_at BEFORE UPDATE ON reviews FOR EACH ROW EXECUTE FUNCTION
            ↪ update_updated_at_column();
    END IF;
END;
```

```
END IF;
END $$;
```

3.7 Notification Service — notification_db

```
-- =====
-- Migration: V1_notification_initial_schema.sql
-- Service: Notification Service
-- =====

CREATE TYPE notification_channel AS ('in_app', 'email', 'sms', 'push');
CREATE TYPE notification_status AS ('unread', 'read', 'archived');

-- Notification templates
CREATE TABLE IF NOT EXISTS notification_templates (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name        VARCHAR(100) NOT NULL UNIQUE,
  subject     VARCHAR(255),
  body_html   TEXT,
  body_text   TEXT,
  channel     notification_channel NOT NULL DEFAULT 'email',
  variables   JSONB NOT NULL DEFAULT '[]',
  is_active   BOOLEAN NOT NULL DEFAULT TRUE,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS idx_notification_templates_name ON notification_templates(name);

-- Notifications
CREATE TABLE IF NOT EXISTS notifications (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID NOT NULL,
  type        VARCHAR(50) NOT NULL, -- course_update, payment, live_class, etc.
  title       VARCHAR(255) NOT NULL,
  message     TEXT NOT NULL,
  data        JSONB, -- payload for deep linking
  channel     notification_channel NOT NULL DEFAULT 'in_app',
  status      notification_status NOT NULL DEFAULT 'unread',
  sent_at     TIMESTAMPTZ,
  read_at     TIMESTAMPTZ,
  expires_at  TIMESTAMPTZ,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS idx_notifications_user_id ON notifications(user_id);
CREATE INDEX IF NOT EXISTS idx_notifications_status ON notifications(status);
CREATE INDEX IF NOT EXISTS idx_notifications_type ON notifications(type);
CREATE INDEX IF NOT EXISTS idx_notifications_created_at ON notifications(created_at);
```

```

-- User notification preferences
CREATE TABLE IF NOT EXISTS user_notification_preferences (
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id       UUID NOT NULL UNIQUE,
    preferences    JSONB NOT NULL DEFAULT '{}',
    created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS idx_unp_user_id ON user_notification_preferences(user_id);

DO $$
BEGIN
    IF NOT EXISTS (SELECT 1 FROM pg_trigger WHERE tgname = 'trg_notification_templates_updated_at') THEN
        CREATE TRIGGER trg_notification_templates_updated_at BEFORE UPDATE ON notification_templates FOR
            ⇨ EACH ROW EXECUTE FUNCTION update_updated_at_column();
    END IF;
    IF NOT EXISTS (SELECT 1 FROM pg_trigger WHERE tgname = 'trg_unp_updated_at') THEN
        CREATE TRIGGER trg_unp_updated_at BEFORE UPDATE ON user_notification_preferences FOR EACH ROW
            ⇨ EXECUTE FUNCTION update_updated_at_column();
    END IF;
END $$;

```

3.8 Admin Service — admin_db

```

-- =====
-- Migration: V1__admin_initial_schema.sql
-- Service: Admin Service
-- =====

-- Audit logs (immutable)
CREATE TABLE IF NOT EXISTS audit_logs (
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    actor_id       UUID NOT NULL, -- admin user id
    actor_type     VARCHAR(20) NOT NULL DEFAULT 'admin' CHECK (actor_type IN ('admin', 'system',
        ⇨ 'student')),
    action         VARCHAR(50) NOT NULL, -- create, update, delete, login, export
    resource_type  VARCHAR(50) NOT NULL, -- course, user, payment, etc.
    resource_id    UUID,
    details        JSONB NOT NULL DEFAULT '{}',
    ip_address     INET,
    user_agent     TEXT,
    created_at     TIMESTAMPTZ NOT NULL DEFAULT NOW()
) PARTITION BY RANGE (created_at);

-- Create monthly partitions for audit_logs (example for first 12 months)
-- In production, use pg_partman or a cron job to create future partitions
CREATE TABLE IF NOT EXISTS audit_logs_2026_08 PARTITION OF audit_logs

```

```

    FOR VALUES FROM ('2026-08-01') TO ('2026-09-01');
CREATE TABLE IF NOT EXISTS audit_logs_2026_09 PARTITION OF audit_logs
    FOR VALUES FROM ('2026-09-01') TO ('2026-10-01');

CREATE INDEX IF NOT EXISTS idx_audit_logs_actor_id ON audit_logs(actor_id);
CREATE INDEX IF NOT EXISTS idx_audit_logs_resource ON audit_logs(resource_type, resource_id);
CREATE INDEX IF NOT EXISTS idx_audit_logs_action ON audit_logs(action);
CREATE INDEX IF NOT EXISTS idx_audit_logs_created_at ON audit_logs(created_at);

-- System settings (key-value with type safety)
CREATE TABLE IF NOT EXISTS system_settings (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    key         VARCHAR(100) NOT NULL UNIQUE,
    value       TEXT NOT NULL,
    data_type   VARCHAR(20) NOT NULL DEFAULT 'string' CHECK (data_type IN ('string', 'number',
        ⇨ 'boolean', 'json')),
    category    VARCHAR(50) NOT NULL DEFAULT 'general', -- general, payment, security, notification
    description  TEXT,
    is_editable BOOLEAN NOT NULL DEFAULT TRUE,
    updated_by  UUID,
    updated_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS idx_system_settings_key ON system_settings(key);
CREATE INDEX IF NOT EXISTS idx_system_settings_category ON system_settings(category);

-- Default settings
INSERT INTO system_settings (key, value, data_type, category, description, is_editable)
VALUES
    ('platform_name', 'Grammarcetamol', 'string', 'general', 'Public platform name', TRUE),
    ('maintenance_mode', 'false', 'boolean', 'general', 'Disable public access', TRUE),
    ('default_currency', 'USD', 'string', 'payment', 'Default currency code', TRUE),
    ('session_timeout_minutes', '60', 'number', 'security', 'Admin session timeout', TRUE),
    ('max_login_attempts', '5', 'number', 'security', 'Lockout threshold', TRUE),
    ('lockout_duration_minutes', '15', 'number', 'security', 'Account lockout duration', TRUE),
    ('password_min_length', '8', 'number', 'security', 'Minimum password length', TRUE),
    ('require_2fa', 'false', 'boolean', 'security', 'Enforce 2FA for admins', TRUE)
ON CONFLICT (key) DO NOTHING;

-- Platform announcements
CREATE TABLE IF NOT EXISTS announcements (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    title       VARCHAR(255) NOT NULL,
    body        TEXT NOT NULL,
    target_audience VARCHAR(50) NOT NULL DEFAULT 'all', -- all, specific_courses, segments
    target_ids  UUID[] NOT NULL DEFAULT '{}',
    priority    VARCHAR(20) NOT NULL DEFAULT 'normal' CHECK (priority IN ('low', 'normal', 'high',
        ⇨ 'critical')),
    status      VARCHAR(20) NOT NULL DEFAULT 'draft' CHECK (status IN ('draft', 'scheduled',
        ⇨ 'published', 'expired')),

```

```

published_at    TIMESTAMPTZ,
expires_at      TIMESTAMPTZ,
published_by    UUID NOT NULL,
created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
updated_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS idx_announcements_status ON announcements(status);
CREATE INDEX IF NOT EXISTS idx_announcements_published_at ON announcements(published_at);
CREATE INDEX IF NOT EXISTS idx_announcements_priority ON announcements(priority);

-- Upload session logs
CREATE TABLE IF NOT EXISTS upload_logs (
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    session_id    UUID NOT NULL,
    course_id     UUID NOT NULL,
    admin_id      UUID NOT NULL,
    status        VARCHAR(20) NOT NULL DEFAULT 'in_progress' CHECK (status IN ('in_progress',
        ↳ 'completed', 'failed', 'abandoned')),
    files_total   INT NOT NULL DEFAULT 0,
    files_success INT NOT NULL DEFAULT 0,
    files_failed  INT NOT NULL DEFAULT 0,
    total_bytes   BIGINT NOT NULL DEFAULT 0,
    uploaded_bytes BIGINT NOT NULL DEFAULT 0,
    error_log     JSONB,
    started_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    completed_at  TIMESTAMPTZ,
    created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS idx_upload_logs_session_id ON upload_logs(session_id);
CREATE INDEX IF NOT EXISTS idx_upload_logs_course_id ON upload_logs(course_id);
CREATE INDEX IF NOT EXISTS idx_upload_logs_admin_id ON upload_logs(admin_id);

DO $$
BEGIN
    IF NOT EXISTS (SELECT 1 FROM pg_trigger WHERE tgname = 'trg_system_settings_updated_at') THEN
        CREATE TRIGGER trg_system_settings_updated_at BEFORE UPDATE ON system_settings FOR EACH ROW
            ↳ EXECUTE FUNCTION update_updated_at_column();
    END IF;
    IF NOT EXISTS (SELECT 1 FROM pg_trigger WHERE tgname = 'trg_announcements_updated_at') THEN
        CREATE TRIGGER trg_announcements_updated_at BEFORE UPDATE ON announcements FOR EACH ROW EXECUTE
            ↳ FUNCTION update_updated_at_column();
    END IF;
END $$;

```

3.9 Upload Service — upload_db

```
-- =====
```

```

-- Migration: V1__upload_initial_schema.sql
-- Service: Upload Service
-- =====

CREATE TYPE upload_status AS ('preparing', 'uploading', 'paused', 'completed', 'failed', 'cancelled');
CREATE TYPE chunk_status AS ('pending', 'uploading', 'completed', 'failed');

-- Upload sessions
CREATE TABLE IF NOT EXISTS upload_sessions (
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    course_id     UUID NOT NULL,
    course_name   VARCHAR(255) NOT NULL,
    admin_id      UUID NOT NULL,
    status        upload_status NOT NULL DEFAULT 'preparing',
    upload_token  VARCHAR(255) NOT NULL UNIQUE,
    chunk_size    INT NOT NULL DEFAULT 5242880, -- 5MB
    max_concurrent INT NOT NULL DEFAULT 5,
    total_files   INT NOT NULL DEFAULT 0,
    completed_files INT NOT NULL DEFAULT 0,
    failed_files  INT NOT NULL DEFAULT 0,
    total_bytes   BIGINT NOT NULL DEFAULT 0,
    uploaded_bytes BIGINT NOT NULL DEFAULT 0,
    expires_at    TIMESTAMPTZ NOT NULL DEFAULT (NOW() + INTERVAL '7 days'),
    started_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    completed_at  TIMESTAMPTZ
);

CREATE INDEX IF NOT EXISTS idx_upload_sessions_course_id ON upload_sessions(course_id);
CREATE INDEX IF NOT EXISTS idx_upload_sessions_admin_id ON upload_sessions(admin_id);
CREATE INDEX IF NOT EXISTS idx_upload_sessions_status ON upload_sessions(status);
CREATE INDEX IF NOT EXISTS idx_upload_sessions_expires_at ON upload_sessions(expires_at);

-- Files within a session
CREATE TABLE IF NOT EXISTS upload_files (
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    session_id    UUID NOT NULL REFERENCES upload_sessions(id) ON DELETE CASCADE,
    lesson_id     UUID,
    file_name     VARCHAR(255) NOT NULL,
    file_size     BIGINT NOT NULL,
    file_type     VARCHAR(50) NOT NULL,
    mime_type     VARCHAR(100),
    status        upload_status NOT NULL DEFAULT 'waiting',
    progress      DECIMAL(5,2) NOT NULL DEFAULT 0.00,
    uploaded_bytes BIGINT NOT NULL DEFAULT 0,
    speed         BIGINT, -- bytes per second
    eta          INT, -- seconds remaining
    checksum      VARCHAR(64) NOT NULL,
    storage_path  VARCHAR(500),
    error_code    VARCHAR(50),
    error_message TEXT,

```

```

    retry_count      INT NOT NULL DEFAULT 0,
    created_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at       TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS idx_upload_files_session_id ON upload_files(session_id);
CREATE INDEX IF NOT EXISTS idx_upload_files_lesson_id ON upload_files(lesson_id);
CREATE INDEX IF NOT EXISTS idx_upload_files_status ON upload_files(status);

-- Chunk tracking
CREATE TABLE IF NOT EXISTS upload_chunks (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    file_id           UUID NOT NULL REFERENCES upload_files(id) ON DELETE CASCADE,
    chunk_index       INT NOT NULL,
    start_byte        BIGINT NOT NULL,
    end_byte          BIGINT NOT NULL,
    size              BIGINT NOT NULL,
    status            chunk_status NOT NULL DEFAULT 'pending',
    checksum           VARCHAR(64) NOT NULL,
    retry_count       INT NOT NULL DEFAULT 0,
    uploaded_at       TIMESTAMPTZ,
    created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE (file_id, chunk_index)
);

CREATE INDEX IF NOT EXISTS idx_upload_chunks_file_id ON upload_chunks(file_id);
CREATE INDEX IF NOT EXISTS idx_upload_chunks_status ON upload_chunks(status);

DO $$
BEGIN
    IF NOT EXISTS (SELECT 1 FROM pg_trigger WHERE tgname = 'trg_upload_sessions_updated_at') THEN
        CREATE TRIGGER trg_upload_sessions_updated_at BEFORE UPDATE ON upload_sessions FOR EACH ROW
            → EXECUTE FUNCTION update_updated_at_column();
    END IF;
    IF NOT EXISTS (SELECT 1 FROM pg_trigger WHERE tgname = 'trg_upload_files_updated_at') THEN
        CREATE TRIGGER trg_upload_files_updated_at BEFORE UPDATE ON upload_files FOR EACH ROW EXECUTE
            → FUNCTION update_updated_at_column();
    END IF;
END $$;

```

4. MongoDB Schema Definitions & Migration Scripts

MongoDB collections are created implicitly on first insert. Migrations focus on **indexes**, **validation rules**, and **data transformations**.

4.1 Media Service — `media_db`

```

// =====
// Migration: 001_media_initial.js

```

```

// Service: Media Service
// Run: mongosh media_db < 001_media_initial.js
// =====

// media.assets collection
// Stores transcoding state, thumbnails, and CDN references
db.createCollection('media_assets', {
  validator: {
    $jsonSchema: {
      bsonType: 'object',
      required: ['fileName', 'fileSize', 'mimeType', 'status', 'createdAt'],
      properties: {
        fileName: { bsonType: 'string' },
        fileSize: { bsonType: 'long' },
        mimeType: { bsonType: 'string' },
        status: { enum: ['uploaded', 'transcoding', 'ready', 'failed', 'deleted'] },
        sourceUrl: { bsonType: 'string' }, // S3/MinIO presigned URL
        variants: {
          bsonType: 'array',
          items: {
            bsonType: 'object',
            properties: {
              resolution: { bsonType: 'string' }, // 720p, 1080p
              url: { bsonType: 'string' },
              format: { bsonType: 'string' }, // hls, mp4
              size: { bsonType: 'long' }
            }
          }
        }
      },
    },
    thumbnailUrl: { bsonType: 'string' },
    duration: { bsonType: 'int' }, // seconds
    metadata: { bsonType: 'object' }, // codec, bitrate, resolution
    courseId: { bsonType: 'binData' }, // UUID as Binary
    lessonId: { bsonType: 'binData' },
    uploadedBy: { bsonType: 'binData' },
    createdAt: { bsonType: 'date' },
    updatedAt: { bsonType: 'date' }
  },
  validationLevel: 'moderate',
  validationAction: 'warn'
});

db.media_assets.createIndex({ courseId: 1, lessonId: 1 });
db.media_assets.createIndex({ status: 1, createdAt: -1 });
db.media_assets.createIndex({ uploadedBy: 1 });
db.media_assets.createIndex({ 'variants.resolution': 1 });

// transcoding.jobs collection
db.createCollection('transcoding_jobs', {

```

```

validator: {
  $jsonSchema: {
    bsonType: 'object',
    required: ['assetId', 'status', 'createdAt'],
    properties: {
      assetId: { bsonType: 'binData' },
      status: { enum: ['queued', 'processing', 'completed', 'failed'] },
      progress: { bsonType: 'int', minimum: 0, maximum: 100 },
      outputs: { bsonType: 'array' },
      error: { bsonType: 'object' },
      startedAt: { bsonType: 'date' },
      completedAt: { bsonType: 'date' },
      createdAt: { bsonType: 'date' }
    }
  }
}
});

db.transcoding_jobs.createIndex({ assetId: 1 }, { unique: true });
db.transcoding_jobs.createIndex({ status: 1, createdAt: 1 });

```

4.2 Live Class Service — `liveclass_db`

```

// =====
// Migration: 001_liveclass_initial.js
// Service: Live Class Service
// =====

db.createCollection('live_classes', {
  validator: {
    $jsonSchema: {
      bsonType: 'object',
      required: ['title', 'instructorId', 'startTime', 'endTime', 'status', 'createdAt'],
      properties: {
        title: { bsonType: 'string' },
        description: { bsonType: 'string' },
        instructorId: { bsonType: 'binData' },
        startTime: { bsonType: 'date' },
        endTime: { bsonType: 'date' },
        timezone: { bsonType: 'string' },
        capacity: { bsonType: 'int', minimum: 1 },
        enrolledCount: { bsonType: 'int', default: 0 },
        price: { bsonType: 'decimal', default: NumberDecimal('0.00') },
        currency: { bsonType: 'string', default: 'USD' },
        meetingUrl: { bsonType: 'string' },
        platform: { enum: ['zoom', 'jitsi', 'google_meet', 'custom'] },
        coverImageUrl: { bsonType: 'string' },
        status: { enum: ['scheduled', 'live', 'completed', 'cancelled'] },
        recordingUrl: { bsonType: 'string' },

```

```

        settings: { bsonType: 'object' }, // mute_on_entry, waiting_room, etc.
        createdBy: { bsonType: 'binData' },
        createdAt: { bsonType: 'date' },
        updatedAt: { bsonType: 'date' }
    }
}
}
});

db.live_classes.createIndex({ instructorId: 1, startTime: 1 });
db.live_classes.createIndex({ status: 1, startTime: 1 });
db.live_classes.createIndex({ startTime: 1 });

db.createCollection('class_bookings', {
    validator: {
        $jsonSchema: {
            bsonType: 'object',
            required: ['classId', 'userId', 'status', 'createdAt'],
            properties: {
                classId: { bsonType: 'binData' },
                userId: { bsonType: 'binData' },
                status: { enum: ['registered', 'attended', 'no_show', 'cancelled'] },
                paymentId: { bsonType: 'binData' },
                joinedAt: { bsonType: 'date' },
                leftAt: { bsonType: 'date' },
                feedback: {
                    bsonType: 'object',
                    properties: {
                        rating: { bsonType: 'int', minimum: 1, maximum: 5 },
                        comment: { bsonType: 'string' }
                    }
                },
                createdAt: { bsonType: 'date' },
                updatedAt: { bsonType: 'date' }
            }
        },
    },
});

db.class_bookings.createIndex({ classId: 1, userId: 1 }, { unique: true });
db.class_bookings.createIndex({ userId: 1, createdAt: -1 });

```

4.3 Analytics Service — analytics_db

```

// =====
// Migration: 001_analytics_initial.js
// Service: Analytics Service
// =====

```

```

// Time-series events (high write volume)
db.createCollection('events', {
  validator: {
    $jsonSchema: {
      bsonType: 'object',
      required: ['eventType', 'userId', 'timestamp', 'payload'],
      properties: {
        eventType: { bsonType: 'string' }, // video_start, lesson_complete, purchase, etc.
        userId: { bsonType: 'binData' },
        sessionId: { bsonType: 'string' },
        timestamp: { bsonType: 'date' },
        payload: { bsonType: 'object' },
        metadata: { bsonType: 'object' } // device, browser, ip, etc.
      }
    }
  }
});

db.events.createIndex({ eventType: 1, timestamp: -1 });
db.events.createIndex({ userId: 1, timestamp: -1 });
db.events.createIndex({ timestamp: 1 }, { expireAfterSeconds: 7776000 }); // TTL 90 days

// Pre-aggregated metrics (dashboard queries)
db.createCollection('daily_metrics', {
  validator: {
    $jsonSchema: {
      bsonType: 'object',
      required: ['date', 'metricType', 'value'],
      properties: {
        date: { bsonType: 'date' },
        metricType: { bsonType: 'string' }, // revenue, enrollments, active_users
        dimension: { bsonType: 'string' }, // course, category, country
        dimensionId: { bsonType: 'binData' },
        value: { bsonType: 'decimal' },
        count: { bsonType: 'int' },
        createdAt: { bsonType: 'date' }
      }
    }
  }
});

db.daily_metrics.createIndex({ date: -1, metricType: 1, dimension: 1 }, { unique: true });

// Search analytics
db.createCollection('search_queries', {
  validator: {
    $jsonSchema: {
      bsonType: 'object',
      required: ['query', 'timestamp'],
      properties: {
        query: { bsonType: 'string' },

```

```

        userId: { bsonType: 'binData' },
        resultsCount: { bsonType: 'int' },
        clickedResult: { bsonType: 'binData' },
        timestamp: { bsonType: 'date' }
    }
}
}
});

db.search_queries.createIndex({ query: 'text' });
db.search_queries.createIndex({ timestamp: -1 });

```

4.4 Service Request Service — request_db

```

// =====
// Migration: 001_servicerequest_initial.js
// Service: Service Request Service
// =====

db.createCollection('service_requests', {
  validator: {
    $jsonSchema: {
      bsonType: 'object',
      required: ['requesterName', 'email', 'serviceType', 'status', 'createdAt'],
      properties: {
        requesterName: { bsonType: 'string' },
        email: { bsonType: 'string' },
        phone: { bsonType: 'string' },
        organization: { bsonType: 'string' },
        serviceType: { bsonType: 'string' }, // corporate_training, consultancy, etc.
        description: { bsonType: 'string' },
        preferredDates: { bsonType: 'array', items: { bsonType: 'date' } },
        budget: { bsonType: 'decimal' },
        status: { enum: ['received', 'under_review', 'responded', 'scheduled', 'completed', 'cancelled']
          ↪ },
        priority: { enum: ['low', 'medium', 'high', 'critical'] },
        assignedTo: { bsonType: 'binData' },
        internalNotes: {
          bsonType: 'array',
          items: {
            bsonType: 'object',
            properties: {
              authorId: { bsonType: 'binData' },
              note: { bsonType: 'string' },
              createdAt: { bsonType: 'date' }
            }
          }
        },
      },
    },
  },
  communicationLog: {

```

```

    bsonType: 'array',
    items: {
      bsonType: 'object',
      properties: {
        type: { enum: ['email', 'status_change', 'internal_note'] },
        from: { bsonType: 'string' },
        to: { bsonType: 'string' },
        subject: { bsonType: 'string' },
        body: { bsonType: 'string' },
        sentAt: { bsonType: 'date' }
      }
    }
  },
  source: { enum: ['website', 'referral', 'social', 'email'] },
  createdAt: { bsonType: 'date' },
  updatedAt: { bsonType: 'date' }
}
}
});

db.service_requests.createIndex({ status: 1, createdAt: -1 });
db.service_requests.createIndex({ assignedTo: 1, status: 1 });
db.service_requests.createIndex({ email: 1 });
db.service_requests.createIndex({ serviceType: 1 });

```

5. Cross-Cutting Concerns

5.1 Foreign Key Strategy Across Services

Since each service owns its database, **no direct foreign keys exist across service boundaries**. Use these patterns:

Relationship	Pattern	Example
Strong Ownership	Local surrogate + event sync	<code>enrollment_db.enrollments.course_id</code> stores UUID; Course Service is source of truth
Lookup / Cache	Read replica or API fallback	User name in <code>payments</code> table fetched from User Service on read
Eventual Consistency	Outbox pattern + RabbitMQ	When a user is deleted, <code>user.deleted</code> event triggers cleanup in all services

5.2 Migration Execution Order

```

01_auth_db/V1__auth_initial_schema.sql
02_user_db/V1__user_initial_schema.sql      -- depends on auth concepts
03_course_db/V1__course_initial_schema.sql  -- depends on users (instructors)

```

```
04_upload_db/V1__upload_initial_schema.sql -- depends on courses
05_enrollment_db/V1__enrollment_initial_schema.sql -- depends on courses + users
06_payment_db/V1__payment_initial_schema.sql -- depends on users + courses
07_review_db/V1__review_initial_schema.sql -- depends on users + courses + enrollments
08_notification_db/V1__notification_initial_schema.sql
09_admin_db/V1__admin_initial_schema.sql
10_mongodb/001_media_initial.js
10_mongodb/001_liveclass_initial.js
10_mongodb/001_analytics_initial.js
10_mongodb/001_servicerequest_initial.js
```

5.3 Running Migrations (Example)

```
# PostgreSQL — using Flyway
flyway -url=jdbc:postgresql://localhost:5432/auth_db -user=admin -password=secret
↪ -locations=filesystem:./migrations/auth_db migrate

# MongoDB — using migrate-mongo (Node)
npx migrate-mongo up
```

5.4 Backup & Disaster Recovery

Layer	Strategy	Frequency
PostgreSQL	pg_dump + WAL archiving	Daily full, continuous WAL
MongoDB	mongodump + Oplog backup	Daily full, hourly incremental
Object Storage (S3/MinIO)	Cross-region replication	Real-time
Redis (Cache)	RDB snapshots + AOF	Every 15 min (ephemeral)

End of Database Schema & Migration Document